



151-0854-00L

Autonomous Mobile Robots

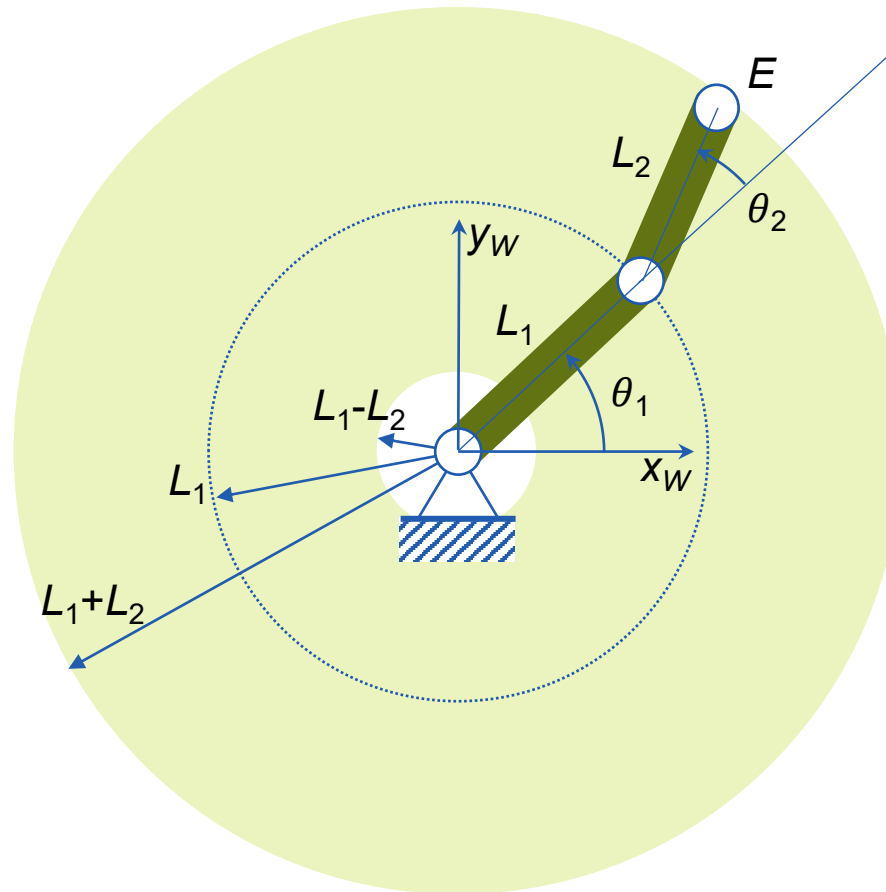
E03: Differential Kinematics & Wheeled Locomotion

Stefan Leutenegger, Anran Zhang, David Rohr



Forward Kinematics: 2R Planar Robot Example

What is the end effector pose for a given configuration? $T_{WE} = f(\theta)$



End effector position / orientation in base-coordinates:

$${}_W\mathbf{t}_{WE} = \begin{bmatrix} L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}.$$
$$\theta_{WE} = \theta_1 + \theta_2$$

Workspace (position) $\mathcal{W}$ for

$$\theta_1 \in [-\pi, \pi], \theta_2 \in [-\pi, \pi].$$

Planar 2R Robot | Inverse Kinematics (IK)

Inverse Kinematics regarding position $\theta({}_W\mathbf{t}_{WE}) = ?$

- ${}_W\mathbf{t}_{WE} \notin \mathcal{W}$: no solution!
- ${}_W\mathbf{t}_{WE} \in \mathcal{W}$: at least one solution.

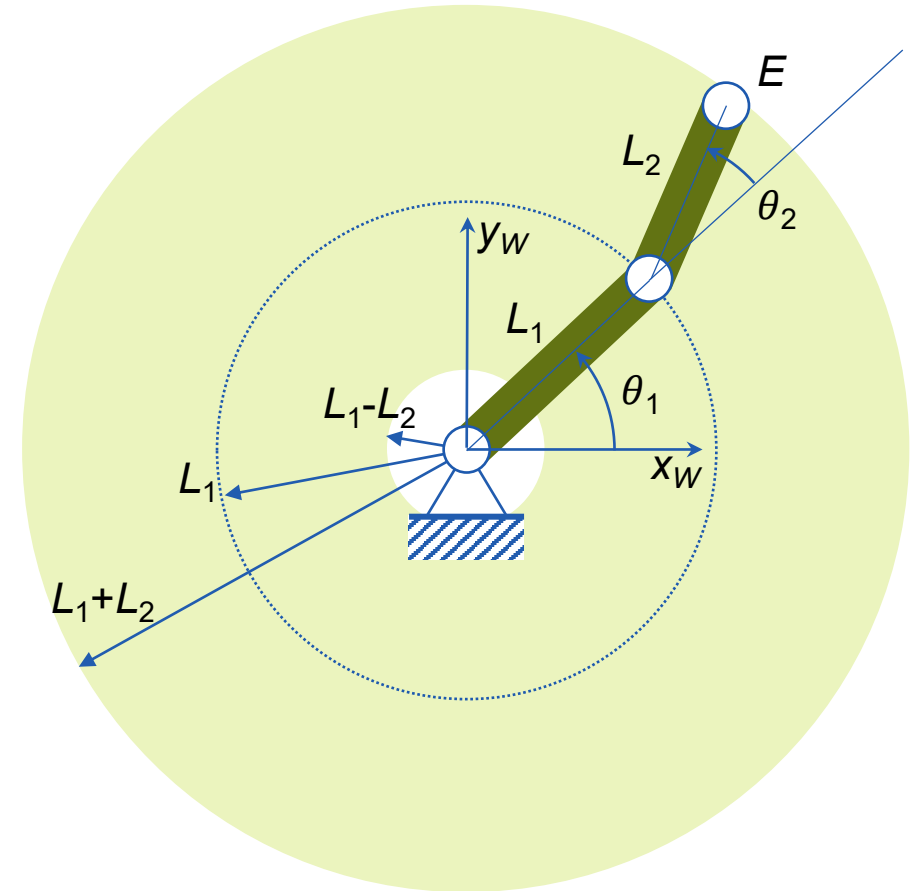
Naive approach:

Solve Forward Kinematics for θ_1, θ_2 :

$$\mathbf{t}_{WE} = \begin{bmatrix} L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\ L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \end{bmatrix}.$$

$\Rightarrow$ Analytically not trivial / feasible in general

$\Rightarrow$ Instead: try iteratively



Velocity Kinematics of Manipulators

Let's derive the **forward velocity kinematics** of a manipulator (end effector) with joints $\theta \in \mathbb{R}^N$ as

$$\begin{bmatrix} {}^W\mathbf{v} \\ {}^W\boldsymbol{\omega} \end{bmatrix} = \underbrace{\mathbf{J}_s(\theta)}_{\text{Manipulator Jacobian}} \dot{\theta},$$

by differentiating

$$\frac{\partial}{\partial t} ({}^W\mathbf{t}_{WE}(\theta)) = {}^W\mathbf{v} = \frac{\partial}{\partial \theta} ({}^W\mathbf{t}_{WE}(\theta)) \dot{\theta},$$

and

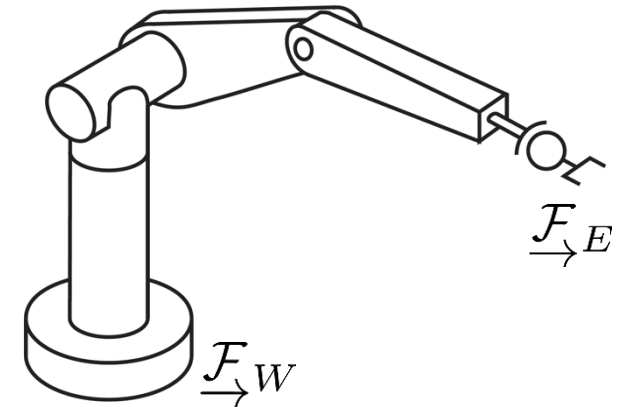
$$\frac{\partial}{\partial t} (\mathbf{R}_{WE}(\theta)) = [{}^W\boldsymbol{\omega}]^\times \mathbf{R}_{WE} = \sum_i \frac{\partial}{\partial \theta_i} (\mathbf{R}_{WE}(\theta_i)) \dot{\theta}_i,$$

$$[{}^W\boldsymbol{\omega}]^\times = \sum_i \frac{\partial}{\partial \theta_i} (\mathbf{R}_{WE}(\theta_i)) \mathbf{R}_{WE}^T \dot{\theta}_i, \quad \text{De-skew operator (vectorise)}$$

$${}^W\boldsymbol{\omega} = \left[\left[\frac{\partial}{\partial \theta_1} (\mathbf{R}_{WE}(\theta_1)) \mathbf{R}_{WE}^T \right]^\mathbf{v}, \dots, \left[\frac{\partial}{\partial \theta_N} (\mathbf{R}_{WE}(\theta_N)) \mathbf{R}_{WE}^T \right]^\mathbf{v} \right] \dot{\theta}.$$

The **inverse velocity kinematics** are now simple

$$\dot{\theta} = \mathbf{J}_s^{-1}(\theta) \begin{bmatrix} {}^W\mathbf{v} \\ {}^W\boldsymbol{\omega} \end{bmatrix}, \text{ or } \dot{\theta} = \mathbf{J}_s^+(\theta) \begin{bmatrix} {}^W\mathbf{v} \\ {}^W\boldsymbol{\omega} \end{bmatrix}, \text{ for overactuated manipulators with } N > 6.$$



Planar 2R Robot | Inverse Kinematics (IK)

Inverse Kinematics regarding position $\theta({}_W\mathbf{t}_{WE}) = ?$

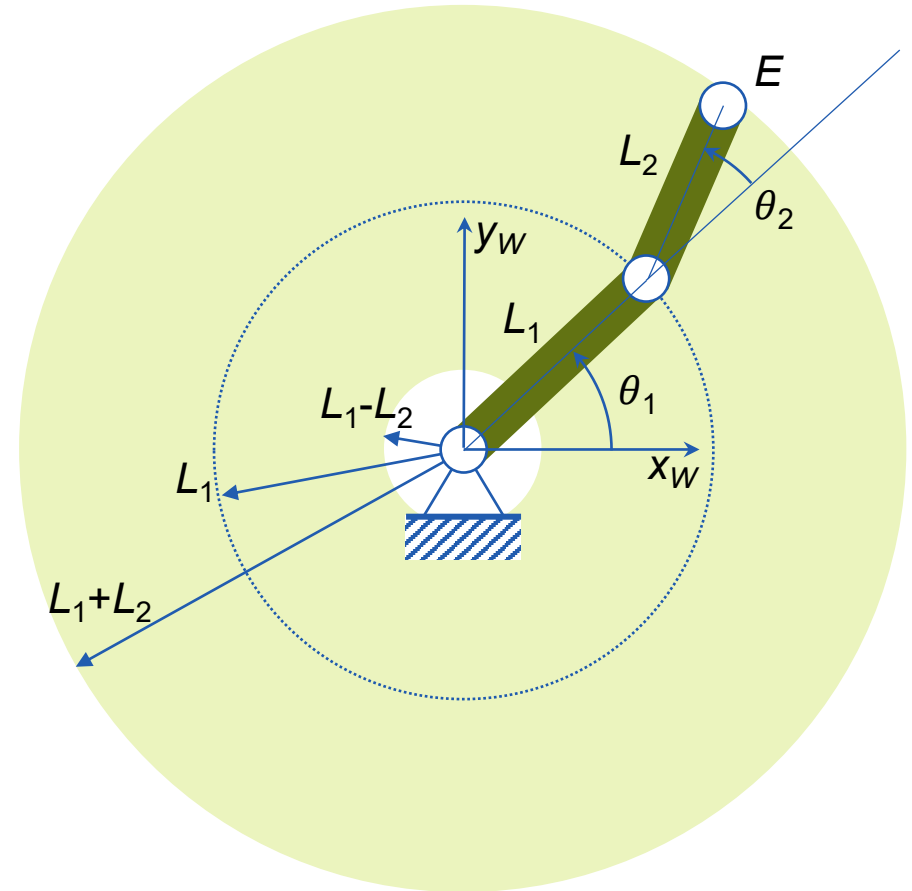
Iterative IK:

- End-effector position Jacobian $\mathbf{J}(\theta)$

$${}_W\mathbf{v}_{WE} = \partial {}_W\mathbf{t}_{WE} / \partial \theta \dot{\theta} := \mathbf{J}(\theta) \dot{\theta}$$

- Iterate until convergence $\|{}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE}\|_2 < \epsilon$

$$\theta \leftarrow \theta + \underbrace{\mathbf{J}(\theta)^\dagger \underbrace{k_p({}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE})}_{\text{incremental step towards goal}}}_{\text{incremental step in joint angles}}$$



Planar 2R Robot | Inverse Kinematics (IK)

Inverse Kinematics regarding position $\theta({}_W\mathbf{t}_{WE}) = ?$

Iterative IK:

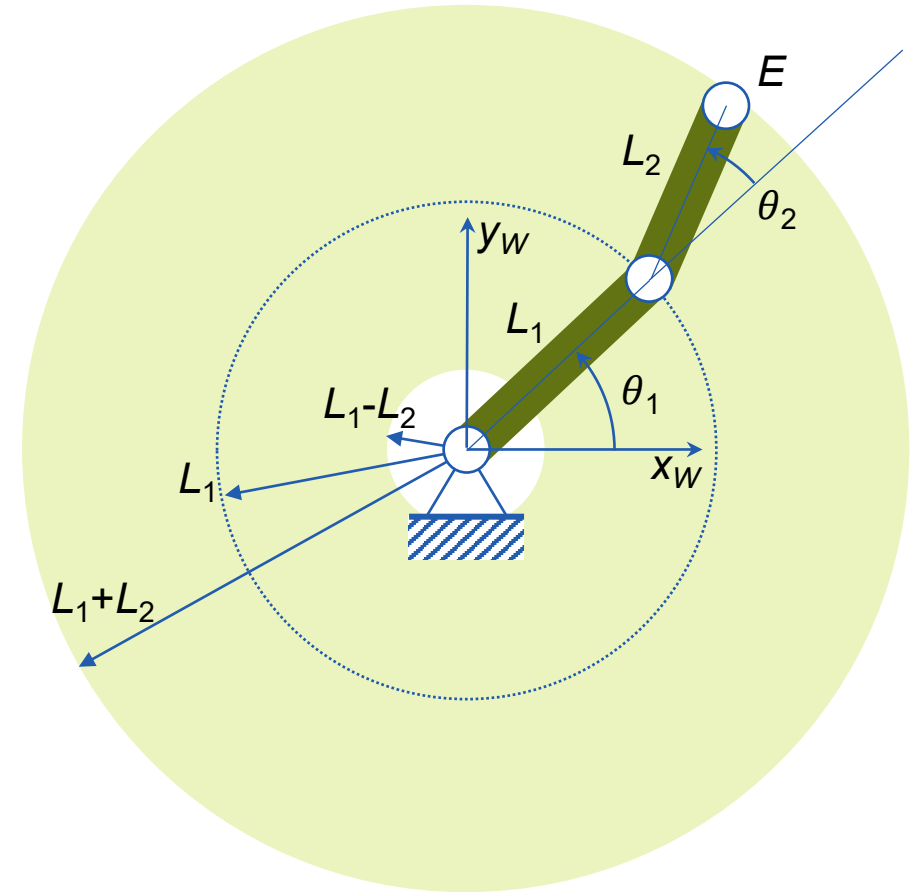
- End-effector position Jacobian $\mathbf{J}(\theta)$

$${}_W\mathbf{v}_{WE} = \partial {}_W\mathbf{t}_{WE} / \partial \theta \dot{\theta} = \mathbf{J}(\theta) \dot{\theta}$$

- Iterate until convergence $\|{}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE}\|_2 < \epsilon$

$$\theta \leftarrow \theta + \mathbf{J}(\theta)^\dagger k_p ({}_W\mathbf{t}_{WE, goal} - {}_W\mathbf{t}_{WE})$$

using pseudoinverse: $\mathbf{J}(\theta)^\dagger = \mathbf{J}(\theta)^T (\mathbf{J}(\theta)\mathbf{J}(\theta)^T)^{-1}$



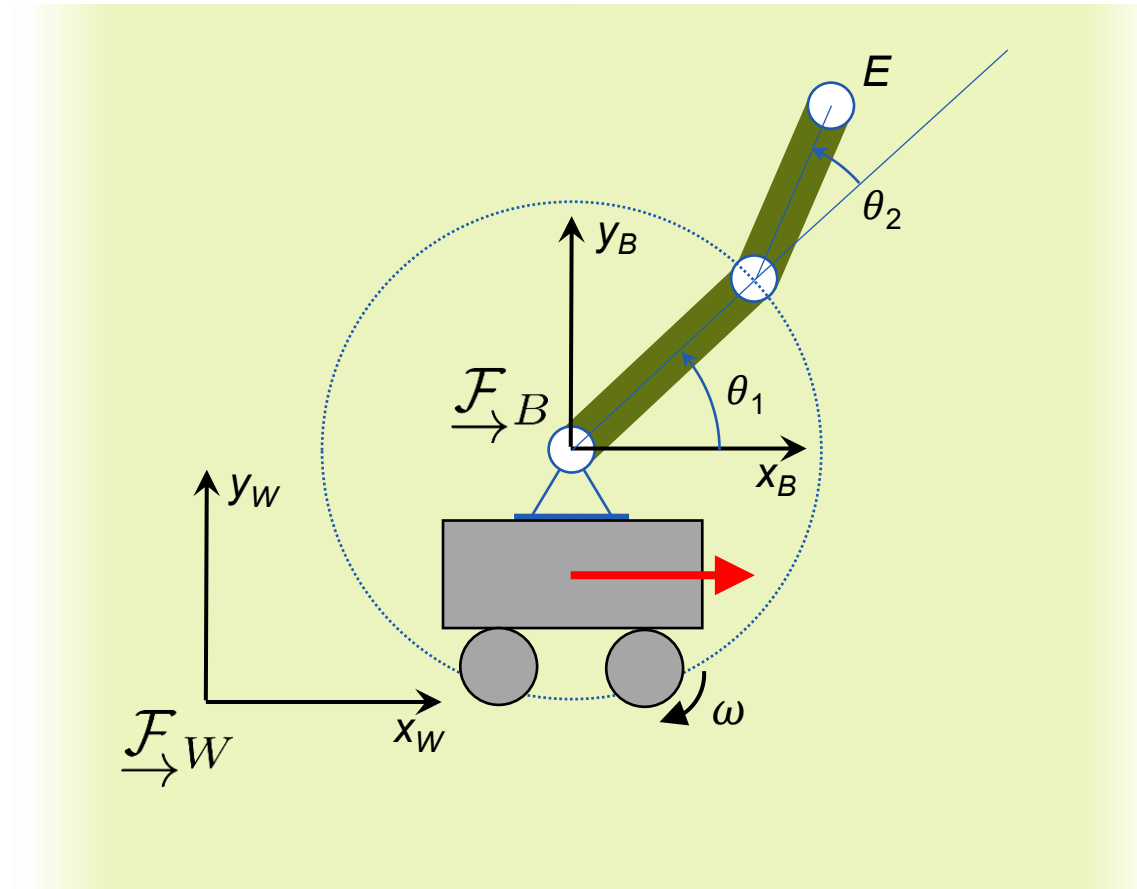
On pseudoinverse for solving $Ax = b$:

- if no sol. exists: $A^\dagger b = \underset{x}{\operatorname{argmin}} \|Ax - b\|_2$
- if mult. sols. exist: $A^\dagger b = \underset{x}{\operatorname{argmin}} \|x\|_2 \text{ s.t. } Ax = b$

Mobile Robot | Forward Kinematics

Actuated base which can move (wheels, legs...)

- extends reachable space



Mobile Robot | Forward Kinematics

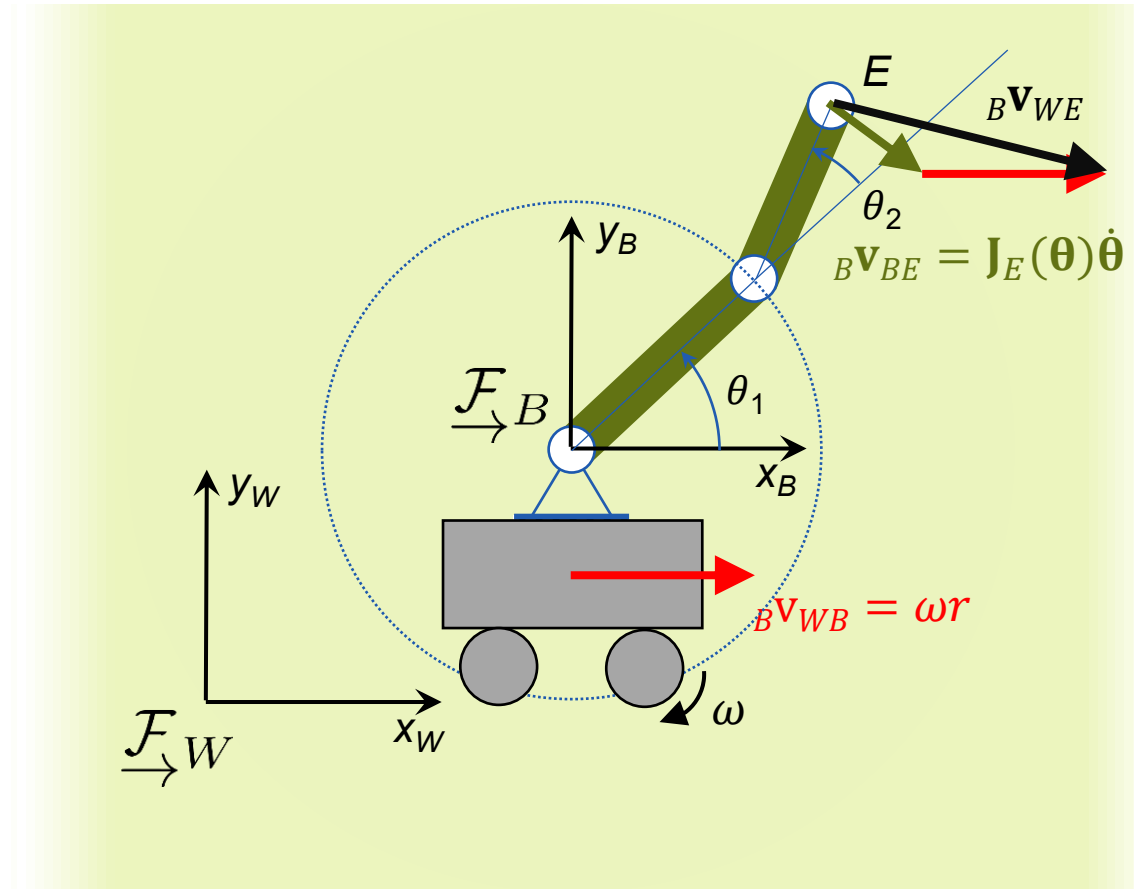
Actuated base which can move (wheels, legs...)

- extends reachable space
- superposition of motions

($\mathbf{u}$:= wheel speeds, ω := angular vel. base)

$${}^W\mathbf{v}_{WE} = {}^W\mathbf{v}_{WB} + {}^W\mathbf{v}_{BE}$$

wrt. world frame



Mobile Robot | Forward Kinematics

Actuated base which can move (wheels, legs...)

- superposition of motions
- extends reachable space

($\mathbf{u}$:= wheel speeds, ω := angular vel. base)

$${}^W\mathbf{v}_{WE} = {}^W\mathbf{v}_{WB} + {}^W\mathbf{v}_{BE}$$

wrt. world frame

$${}^B\mathbf{v}_{WE} = {}^B\mathbf{v}_{WB} + {}^B\boldsymbol{\omega} \times {}^B\mathbf{t}_{BE} + {}^B\dot{\mathbf{t}}_{BE}$$

wrt. base frame

$$= \mathbf{J}_E^{\mathbf{u}}(\boldsymbol{\theta})\mathbf{u} + \mathbf{J}_E^{\dot{\boldsymbol{\theta}}}(\boldsymbol{\theta})\dot{\boldsymbol{\theta}}$$

base & end-eff. Jacobian

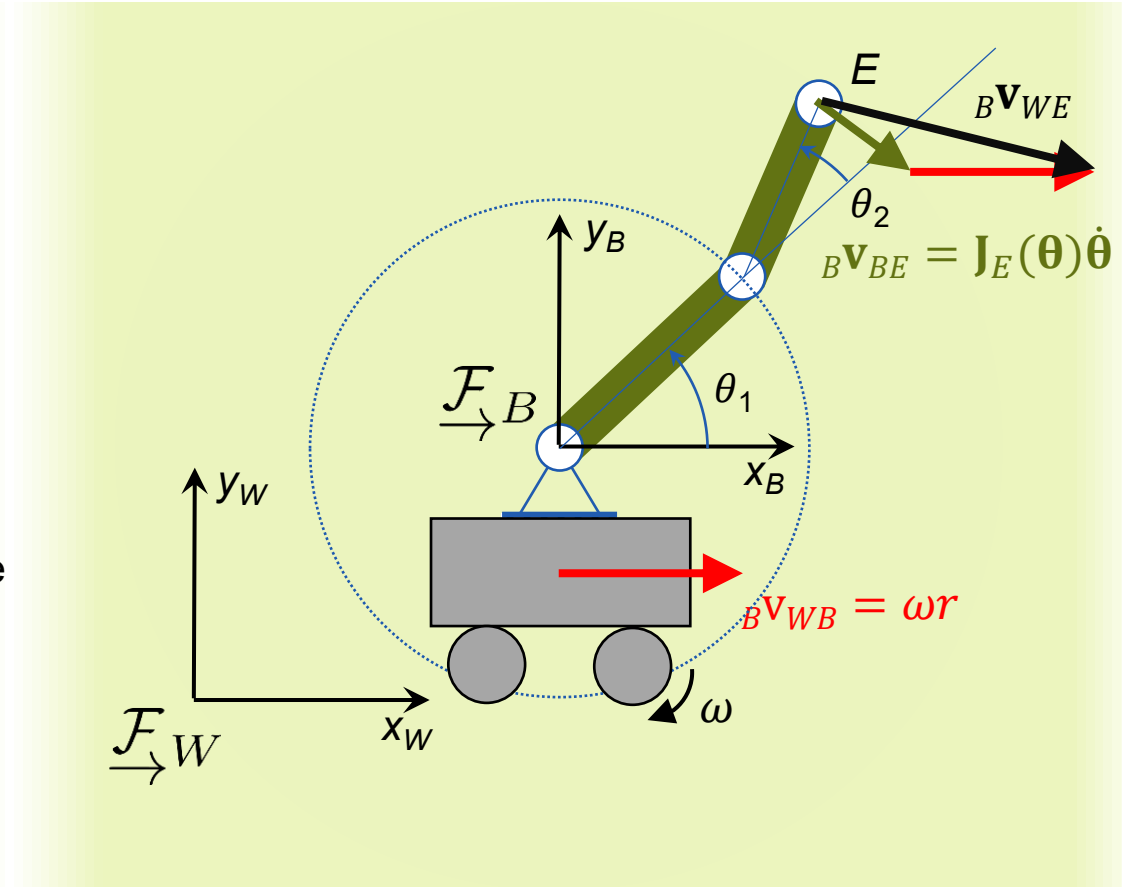
$$= [\mathbf{J}_E^{\mathbf{u}}(\boldsymbol{\theta}), \mathbf{J}_E^{\dot{\boldsymbol{\theta}}}(\boldsymbol{\theta})] \begin{bmatrix} \mathbf{u} \\ \dot{\boldsymbol{\theta}} \end{bmatrix}$$

→ additional actuators/joints, extended configuration space $[\dot{\boldsymbol{\theta}}, \mathbf{u}]$, with $\mathbf{u}$:= wheel speeds.

→ apply iterative IK to reach / track a target using base + arm

task 3.1

→ how to get $\mathbf{J}_E^{\mathbf{u}}(\boldsymbol{\theta})$?



Mobile Robot | Wheel Constraints

How to get $\mathbf{J}_E^u(\theta)$, relating wheel rot. speed and end-eff. vel.? ${}^B\mathbf{v}_{WE} = [\mathbf{J}_E^u(\theta), \mathbf{J}_E^{\dot{\theta}}(\theta)] \begin{bmatrix} \mathbf{u} \\ \dot{\theta} \end{bmatrix}$

1. Base twist ${}^B\mathbf{v}_B = [{}^B\mathbf{v}_B^T, \Omega]^T$ to wheel-hub velocity ${}^B\mathbf{v}_{D_i}$

$${}^B\mathbf{v}_{D_i} = {}^B\mathbf{v}_B + \begin{bmatrix} 0 \\ 0 \\ \Omega \end{bmatrix} \times {}^B\mathbf{t}_{BD_i} \quad \text{planar robot example}$$

2. Admissible base twist has to adhere to wheel constraints, i.e.

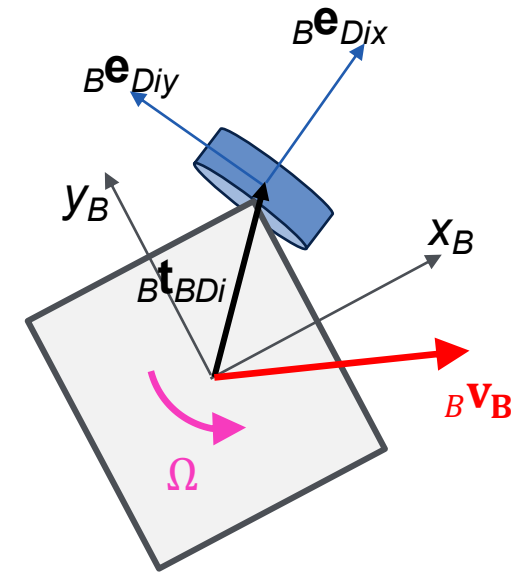
$$\begin{aligned} {}^B\mathbf{v}_{D_i} \cdot {}^B\mathbf{e}_{D_i x} &= 0 & \text{sliding constraint} \\ {}^B\mathbf{v}_{D_i} \cdot {}^B\mathbf{e}_{D_i y} &= -\omega_i r_i & \text{rolling constraint} \end{aligned}$$

3. Form rolling constraint matrix $\mathbf{C}_r$: $\mathbf{C}_r {}^B\mathbf{v}_B = \mathbf{u}$ with $\mathbf{u} := [\omega_0 \dots \omega_{N-1}]$ wheel speeds and invert (assuming possible) as

$$\underbrace{{}^B\mathbf{v}_{WB} + \begin{bmatrix} 0 \\ 0 \\ \Omega \end{bmatrix} \times {}^B\mathbf{t}_{BE}(\theta)}_{\mathbf{A}(\theta)} = \mathbf{A}(\theta) {}^B\mathbf{v}_B = \mathbf{A}(\theta) \mathbf{C}_r^{-1} \mathbf{u} = \mathbf{J}_E^u(\theta) \mathbf{u}$$

$\mathbf{A}(\theta)$ maps twist to end-eff. velocity in body frame

cf. hint in task 2.4

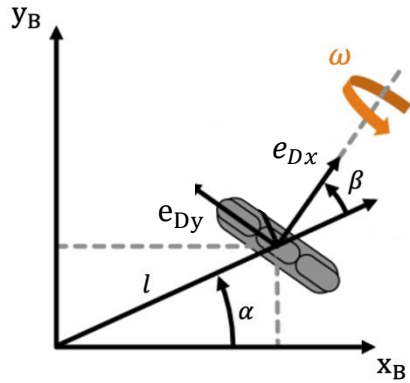


task 1.2

task 2.2, where $\mathbf{J}_B := \mathbf{C}_r^{-1}$

task 2.4

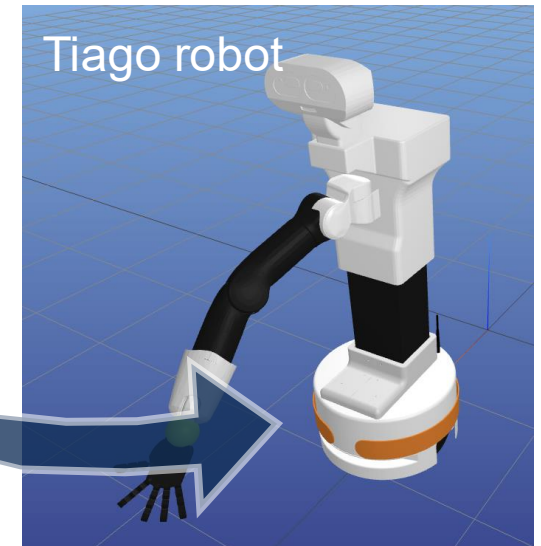
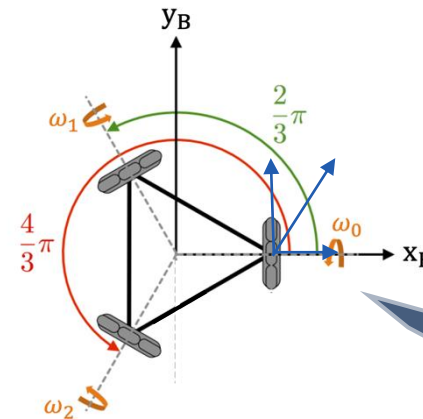
Mobile Robot | Exercise



- 90 degree Swedish wheel

- 3-wheels omnidirectional base
- Integration with Tiago robot

$$\mathbf{u} := \begin{bmatrix} \omega_0 \\ \omega_1 \\ \omega_2 \end{bmatrix}$$



Thanks! Questions?

E02: Python environment setup

Via Conda:

- <https://www.anaconda.com/docs/getting-started/miniconda/install#quickstart-install-instructions>
- `source ~/miniconda3/bin/activate && conda init --all`, restart terminal

Task	Windows (Anaconda/Miniforge Prompt)	macOS / Linux (Terminal)
Create Env	<code>conda create -n myenv python=3.10</code>	<code>conda create -n myenv python=3.10</code>
Activate	<code>conda activate myenv</code>	<code>conda activate myenv</code> (or <code>source activate myenv</code>)
Deactivate	<code>conda deactivate</code>	<code>conda deactivate</code>
List Envs	<code>conda env list</code>	<code>conda env list</code>
Install	<code>conda install yourpackages -c conda-forge</code>	<code>pip install -r requirements.txt</code>

E02: Python environment setup

Via virtual environment (venv):

Feature	Windows	macOS	Linux (Ubuntu/Debian)
Command Prefix	<code>python</code> or <code>py</code>	<code>python3</code>	<code>python3</code>
Create Cmd	<code>python -m venv .venv</code>	<code>python3 -m venv .venv</code>	<code>python3 -m venv .venv</code>
Activate Cmd	<code>.venv\Scripts\activate</code>	<code>source .venv/bin/activate</code>	<code>source .venv/bin/activate</code>
Pre-installed?	Yes (if installed via python.org)	Yes (via Xcode Tools)	No (requires <code>apt install</code>)
Extra Step	May need Execution Policy fix	None	<code>sudo apt install python3-venv</code>
Verify Path	<code>where python</code>	<code>which python</code>	<code>which python</code>
Deactivate	<code>deactivate</code>	<code>deactivate</code>	<code>deactivate</code>
Install	<code>pip install -r requirements.txt</code>	<code>pip install -r requirements.txt</code>	<code>pip install -r requirements.txt</code>

E02: Python environment setup

Jupyter Notebooks

Run directly in VS Code (Jupyter extension)

OR

`pip/conda install notebook && jupyter notebook` (launches a window in browser, then select file and go through the blocks. Can run blocks with shift+enter)